

UUV MuJoCo v2.2 상세 설명 보고서

지금 코드만 보고도 구조와 물리를 설명할 수 있게 정리한 독립 문서

2026-04-05

Contents

1	이 문서의 목적	3
2	한 장 요약	3
3	물리엔진은 한 step에서 어떻게 모델링되는가	3
3.1	실제로는 어떤 순서로 계산되는가	4
3.2	좌표계는 왜 중요한가	4
3.3	current model은 정확히 무엇이 MuJoCo이고 무엇이 Python인가	5
3.4	왜 이렇게 하이브리드 구조로 만들었는가	6
4	용어를 모르면 먼저 여기부터	6
5	현재 active 파일은 무엇이고, 각각 무엇을 바꾸는가	7
6	현재 로봇 형상과 좌표계는 어떻게 배치되어 있는가	7
6.1	장면의 핵심은 body 하나와 site 여러 개다	7
6.2	3D로 보면 무엇이 한 번에 보이는가	7
6.3	위에서 보면 무엇이 보이는가	8
6.4	옆에서 보면 무엇이 보이는가	8
7	6DOF에서 어떤 물리력이 어떤 축에 들어가는가	9
7.1	먼저 body 축과 상태벡터를 읽어야 한다	9
7.2	custom 경로에서는 어떤 힘을 합쳐서 최종 wrench를 만드는가	10
7.3	어떤 향이 어떤 자유도에 주로 작용하는가	11
7.4	이전 custom baseline과 현재 ellipsoid 모델은 무엇이 다르게 켜지는가	12
8	질량, 관성, 부력은 현재 어떻게 구현되어 있는가	13
8.1	현재 질량 모델	13
8.2	현재 관성 모델	13
8.3	현재 부력 모델	14
9	스러스터 모델은 현재 어떻게 구현되어 있는가	15

9.1	현재 공통 thruster knob	15
9.2	실제 추력 생성 경로	16
10	이전 custom baseline과 현재 ellipsoid 모델의 hydrodynamics는 어떻게 다른가	16
10.1	이전 custom baseline	16
10.2	현재 ellipsoid 모델	17
10.3	profile에서 runtime으로 파생되는 hydrodynamic 값	17
11	왜 MANUAL yaw와 DEPTH_HOLD yaw 느낌이 다를 수 있는가	18
12	현재 측정 결과는 무엇을 말해 주는가	19
12.1	실제 mode path 비교	19
12.2	저장돼 있는 archived engine snapshot	19
13	현재 문제를 볼 때 어디를 먼저 보면 되는가	20
14	실행과 재현 방법	21
14.1	권장 실행	21
14.2	새 문서 자산 생성	21
14.3	새 PDF 빌드	22
15	이 문서만 보고 바로 설명할 때의 말하기 순서	22
16	결론	22

1 이 문서의 목적

이 문서는 **현재 repository의 active 코드 상태**를 기준으로, MuJoCo와 로봇/역학에 익숙하지 않은 사람도 바로 설명할 수 있도록 만든 **독립형 설명 문서**다. 기존 `mujoco_fluid_forces_kr.tex` 를 수정한 것이 아니라, 2026년 4월 5일 시점의 `uuv_mujoco/v2.2` 스냅샷을 별도 보고서로 다시 구성했다.

이 문서가 답하려는 질문은 단순하다.

1. 지금 시스템은 어떤 파일들이 어떻게 연결되어 움직이는가.
2. 로봇의 질량, 부력, 유체 저항, 스러스터, FCU 제어가 각각 어디에서 구현되는가.
3. 이전 custom baseline과 현재 ellipsoid 기반 모델은 무엇이 다르고, 실제 측정 결과는 어떻게 다른가.
4. 특정 이상 현상(느린 yaw, 깊이 흔들림, pitch bias, 과한 damping)이 보일 때 어디를 먼저 봐야 하는가.

2 한 장 요약

이 repository에는 두 runtime path가 남아 있지만, 설명의 기준 모델은 하나다.

- custom: 예전에 쓰던 Python 직접 계산형 6-DOF hydrodynamics baseline. 비교와 회귀 확인을 위해 남겨 둔 이전 모델이다.
- ellipsoid: **현재 모델**. scene의 `fluidshape="ellipsoid"` 와 `fluidcoef` 를 이용한 MuJoCo built-in ellipsoid fluid 를 바탕으로, runtime에서 분산 질량/분산 부력/heave damping/reaction torque 같은 보정을 함께 얹는다.

즉 현재 조종감과 실제 비교의 기준은 ellipsoid 하나이고, custom 은 “이전에는 이렇게 모델링했었다”를 설명하는 baseline으로 읽는 것이 맞다.

MuJoCo는 강체 적분기이자 센서/scene 엔진이고, ArduSub는 제어기이며, JSON/XML은 물리 파라미터 파일이다.

- ArduSub는 “얼마나 돌려라, 얼마나 내려가라” 를 결정한다.
- MuJoCo runtime은 그 명령을 실제 힘으로 바꾼다.
- `sim_profiles.json` 은 부력, drag, added mass, 관성 스케일을 정한다.
- `thruster_params.json` 은 스러스터의 데드존, lag, 전역 gain, 반전 비대칭을 정한다.
- `tank_ellipsoid_scene.xml` 은 body, 센서 site, thruster site, built-in fluid proxy의 위치를 정한다.

3 물리엔진은 한 step에서 어떻게 모델링되는가

이 문서에서 말하는 “물리엔진”은 사실 두 층으로 나뉜다.

- **Python runtime**: 어떤 힘과 모멘트를 만들어 MuJoCo에 넣을지 계산한다.
- **MuJoCo solver**: 그 힘과 모멘트, actuator, contact, built-in fluid를 받아 실제 강체 상태를 적분한다.

즉 현재 구현은 “MuJoCo가 모든 유체력을 혼자 계산한다”도 아니고, “Python이 모든 physics를 다 적분한다”도 아니다. **Python이 wrench를 만들고, MuJoCo가 적분한다**가 가장 정확한 설명이다.

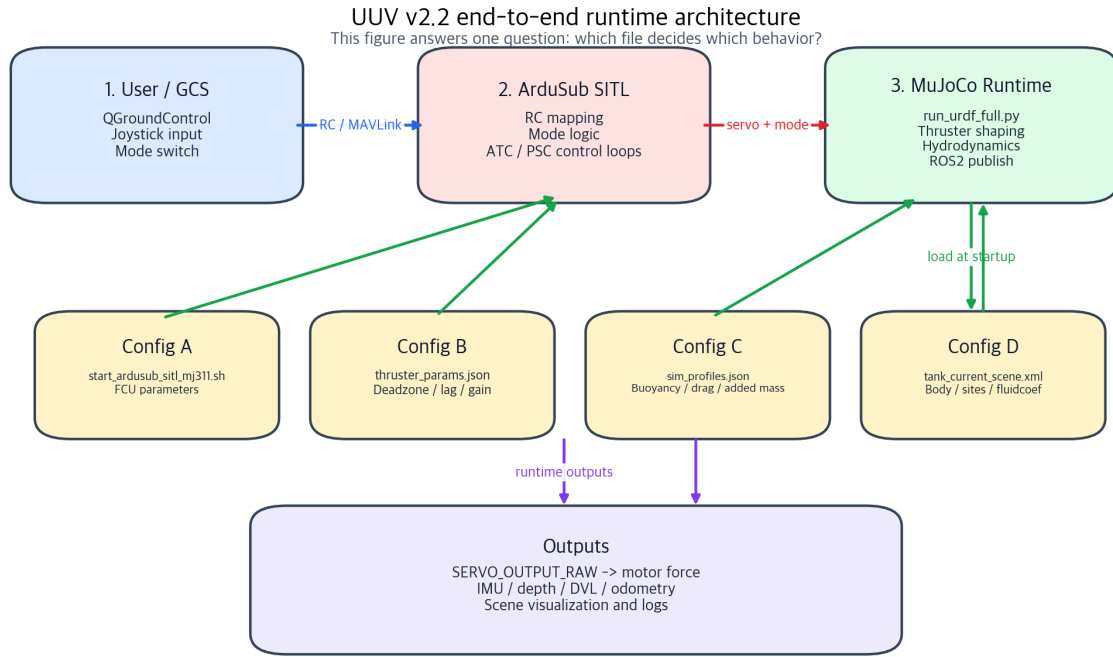


Figure 1: 현재 v2.2 전체 구조. 이 그림 하나만 보면 “어떤 파일이 어떤 역할을 하는지”를 설명할 수 있다.

3.1 실제로는 어떤 순서로 계산되는가

한 step의 개념적 순서는 다음과 같다.

1. MuJoCo state에서 현재 body pose, body/world velocity, depth, rotation matrix를 읽는다.
2. FCU 또는 SITL servo 입력을 받아 deadzone, saturation, lag, force curve를 거쳐 `data.ctrl` 를 만든다.
3. buoyancy point 또는 body component를 기준으로 잠김비를 계산하고, 분산 부력과 CoM/CoB torque를 만든다.
4. 수면 근처 또는 완전 잠수 구간 heave damping을 더한다.
5. 이전 baseline인 custom 경로라면 added mass, added-mass Coriolis, 선형 감쇠, 2차 감쇠로 body-frame hydrodynamic wrench를 계산한다.
6. 현재 모델인 ellipsoid 경로라면 위 custom wrench는 끄고, 대신 MuJoCo built-in ellipsoid fluid가 scene의 proxy geom들에서 drag를 계산하게 둔다.
7. thruster reaction torque, yaw torque scale 같은 보조 토크를 더한다.
8. Python이 만든 wrench를 `data.xfrc_applied` 에 넣고, actuator scalar는 `data.ctrl` 에 넣는다.
9. 마지막에 MuJoCo가 `mj_step(model, data)` 를 수행해 강체 적분, actuator 적용, built-in fluid/contact/constraint 해결을 한다.

3.2 좌표계는 왜 중요한가

여기서 가장 중요한 구현 포인트는 **body frame**과 **world frame**을 섞어 쓰지 않는 것이다.

- hydrodynamics helper가 쓰는 속도 상태는 bodyframe의 $\nu = [u, v, w, p, q, r]^T$ 이다.
- distributed buoyancy force는 계산 결과가 worldz 방향 힘으로 들어간다.

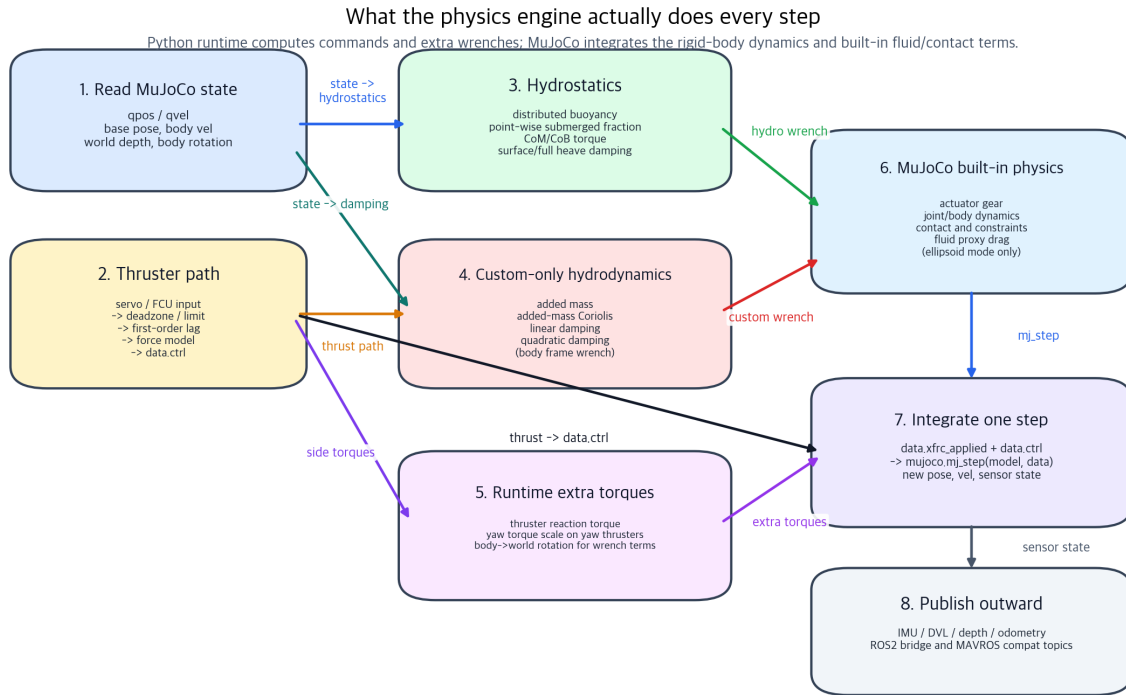


Figure 2: runtime 한 step의 실제 순서. Python runtime이 `data.ctrl` 와 `data.xfrc_applied` 를 준비하고, MuJoCo가 `mj_step` 으로 강체 적분과 built-in physics를 수행한다.

- custom hydrodynamic wrench는 먼저 `bodyframe`에서 계산한 뒤, `base rotation`으로 `world frame`으로 돌려 `xfrc_applied` 에 넣는다.
- MuJoCo의 `xfrc_applied` 는 `[force_xyz, torque_xyz]` `world frame` 6-vector 이다.

즉 “어떤 항을 어느 좌표계에서 계산하고, 마지막에 어디로 넣느냐”가 구현의 핵심이다. 이 부분을 놓치면 `depth hold` 중 수직 추력 이 왜 수평 drift로 새는지 같은 현상을 설명할 수 없다.

3.3 current model은 정확히 무엇이 MuJoCo이고 무엇이 Python인가

현재 모델(ellipsoid)을 기준으로 나누면 다음과 같다.

항목	Python runtime이 담당	MuJoCo가 담당
Distributed mass / inertia synthesis	<code>body_components</code> 를 읽어 합성 CoM / inertia 계산	합성된 mass / inertia를 가진 강체 적분
Thruster shaping	deadzone, command limit, lag, force scaling, yaw torque scale, reaction torque	actuator gear direction과 scalar 적용
Buoyancy	submerged fraction, point-wise buoyancy, buoyancy torque 계산	주어진 force/torque로 rigid body 반응
Custom hydrodynamics	added mass, Coriolis, linear/quadratic damping wrench 계산	없음
Ellipsoid fluid drag	없음	scene의 <code>fluid_*</code> proxy geom에서 built-in ellipsoid fluid 계산
Rigid-body integration	없음	<code>mj_step</code> 으로 pose/velocity/contact 적분
Sensor state	publish용 변환 일부	실제 body 상태와 sensor frame 기반 원 상태 제공

3.4 왜 이렇게 하이브리드 구조로 만들었는가

이 구조를 택한 이유는 단순하다.

- **MuJoCo만으로는** 분산 부력, custom added-mass Coriolis, runtime-specific yaw authority 같은 세밀한 수중 모델을 설명하기 어렵다.
- **Python만으로는** 안정적인 강체 적분, actuator, contact, sensor frame, visualization을 다시 만들 필요가 생긴다.
- 그래서 현재 구현은 “수중 전용 force model은 Python에서 만들고, 최종 dynamics integration은 MuJoCo에 맡기는” 형태로 정리되어 있다.

4 용어를 모르면 먼저 여기부터

용어	뜻
질량 (mass)	병진 운동의 무거움을 정하는 값. 크면 앞뒤/좌우/상하로 움직이기 어려워진다.
관성 (inertia)	회전 운동의 무거움을 정하는 값. 크면 roll/pitch/yaw가 시작도 느리고 멈추는 데도 오래 걸린다.
CoM	Center of Mass. 질량중심. 무게가 대표적으로 작용하는 점.
CoB	Center of Buoyancy. 부심. 부력이 대표적으로 작용하는 점.
부력 (buoyancy)	잠긴 부피 때문에 생기는 위쪽 힘. 기본식은 $F_B = \rho g V_{\text{sub}}$.
added mass	물속에서 주변 유체도 같이 가속해야 해서 생기는 추가 관성. 수중체가 공기 중보다 더 무겁게 느껴지는 원인 중 하나다.
damping / drag	이미 움직이는 물체를 줄여 주는 저항. 크면 명령을 끝냈을 때 빨리 정리된다.
thruster gain	조이스틱/FCU 명령이 실제 추력으로 얼마나 크게 바뀌는지 정하는 스케일.
lag (τ)	모터 명령이 즉시 힘으로 나오지 않고 지연되어 따라오는 정도.
fluidcoef	MuJoCo built-in ellipsoid fluid proxy에 주는 계수 묶음. drag와 angular drag 체감을 크게 바꾼다.

5 현재 active 파일은 무엇이고, 각각 무엇을 바꾸는가

파일	역할
uuv_mujoco/v2.2/launch_uuv_sim.sh	MuJoCo runtime 실행 wrapper. scene 선택, fluid model 선택, ROS2/QGC bridge 옵션을 넘긴다.
uuv_mujoco/v2.2/reset_uuv_sim.sh	MuJoCo, SITL, MAVProxy, QGC, 외부 MAVROS 잔존 프로세스를 정리한다.
uuv_mujoco/v2.2/start_sitl_mujoco_mj311.sh	reset 후 SITL과 MuJoCo를 같이 띄운다. 로그 디렉터리도 여기서 만든다.
uuv_mujoco/v2.2/start_ardusub_sitl_mj311.sh	ArduSub SITL 파라미터를 주입한다. yaw/depth/attitude PID와 RC deadzone도 여기서 바뀐다.
uuv_mujoco/v2.2/run_urdf_full.py	실제 runtime 핵심. thruster shaping, buoyancy, hydrodynamics, distributed mass/inertia, ROS2 bridge를 모두 처리한다.
uuv_mujoco/v2.2/config/sim_profiles.json	이전 baseline(custom)과 현재 모델(ellipsoid)의 물리 knob 모음. 부력, drag, added mass, CoB offset, inertia scale이 여기 있다.
uuv_mujoco/v2.2/config/thruster_params.json	스러스터 공통 deadzone, lag, global gain, 반전 비대칭, polynomial force curve가 여기 있다.
uuv_mujoco/v2.2/config/thruster_performance.json	T200 PWM-force curve 원본 데이터. 전압별 실제 측정 곡선이다.
uuv_mujoco/v2.2/scenes/tank_custom_scene.xml	이전 baseline인 custom hydrodynamics 경로용 scene. 비교/회귀 확인용으로 남아 있다.
uuv_mujoco/v2.2/scenes/tank_ellipsoid_scene.xml	현재 모델 의 핵심 scene. built-in ellipsoid fluid와 site, sensor, thruster, fluid proxy geom을 정의한다.
uuv_mujoco/v2.2/physics/sim_profile_helpers.py	JSON profile을 runtime이 쓰기 쉬운 dataclass로 바꾸고, ellipsoid baseline을 계산한다.
uuv_mujoco/v2.2/physics/hydrodynamics_helpers.py	잠김비, thruster shaping, polynomial force, 6-DOF hydrodynamics helper 함수가 있다.
uuv_mujoco/v2.2/physics/thruster_mapping.py	thruster 순서, ArduSub servo mapping, sensor FRD/FLU 기준을 정의한다.

6 현재 로봇 형상과 좌표계는 어떻게 배치되어 있는가

6.1 장면의 핵심은 body 하나와 site 여러 개다

현재 tank_ellipsoid_scene.xml 의 base_link body 아래에는 세 부류의 위치 정보가 있다.

- **thruster site**: 스러스터가 힘을 걸어 주는 위치
- **mass site**: 분산 질량 모델을 설명하기 위한 질량 component 위치
- **buoyancy site**: 분산 부력 모델을 설명하기 위한 부력 point 위치

여기서 중요한 점은, **scene XML의 mesh 자체가 질량을 계산하는 것이 아니라**, runtime이 JSON의 body_components 를 읽어 **합성 CoM과 합성 inertia** 를 다시 계산한다는 것이다. 즉 mesh는 모양, JSON은 물리 파라미터, Python runtime은 실제 합성 계산기다.

6.2 3D로 보면 무엇이 한 번에 보이는가

이 그림 하나로 바로 설명할 수 있는 것은 다음과 같다.

- **질량은 어디에 분산되어 있는가**: 빨간 점 세 개가 중앙 인클로저와 좌우 하부 body의 질량 component다.
- **부력은 어디에 분산되는가**: 초록 점 네 개가 vertical thruster 바로 위의 buoyancy point다.

3D isometric layout: fluid proxy geoms, distributed mass, buoyancy points, sensors, and thruster directions

This is the most direct picture for explaining trim, pitch bias, CoM/CoB offset, and why thrust planes matter.

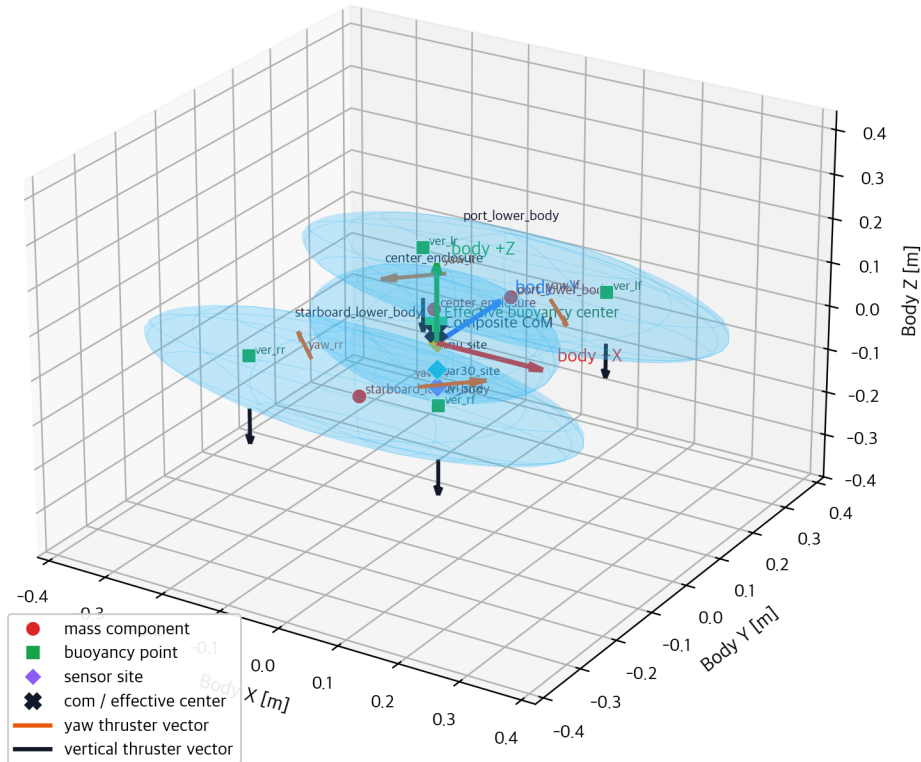


Figure 3: 3D isometric layout. built-in ellipsoid fluid proxy geom, distributed mass component, buoyancy point, 센서 위치, thruster vector, body 축을 한 좌표계에 겹쳐 놓은 그림이다.

- 수직 추력은 어디서 올라오는가: 검정 벡터가 vertical thruster plane을 보여 준다.
- yaw 토크는 어떻게 만들어지는가: 주황 벡터가 대각선 yaw thruster 방향을 보여 준다.
- 왜 trim bias가 생기는가: CoM, 유효 buoyancy center, fluid proxy 중심, thruster plane의 상대 위치가 한 화면에서 보인다.

6.3 위에서 보면 무엇이 보이는가

이 그림에서 바로 읽을 수 있는 핵심은 다음과 같다.

- 로봇은 중앙 인클로저 1개 + 좌우 하부 body 2개 의 3분할 질량 모델을 쓴다.
- 부력은 중앙 한 점이 아니라 수직 슬러스터 바로 위 4점에 나눠 준다.
- yaw thruster는 대각선 방향으로 놓여 있어, 순수 전진/순수 sway뿐 아니라 회전 모멘트도 만든다.
- 현재 유효 buoyancy center는 CoM의 x축과 거의 맞춘 상태다. 이걸 depth hold 중 뒤로 밀리는 trim bias를 줄이기 위한 의도적인 정렬이다.

6.4 옆에서 보면 무엇이 보이는가

측면도에서 읽어야 할 것은 다음과 같다.

- 깊이 유지 추력은 vertical thruster plane에서 나온다.

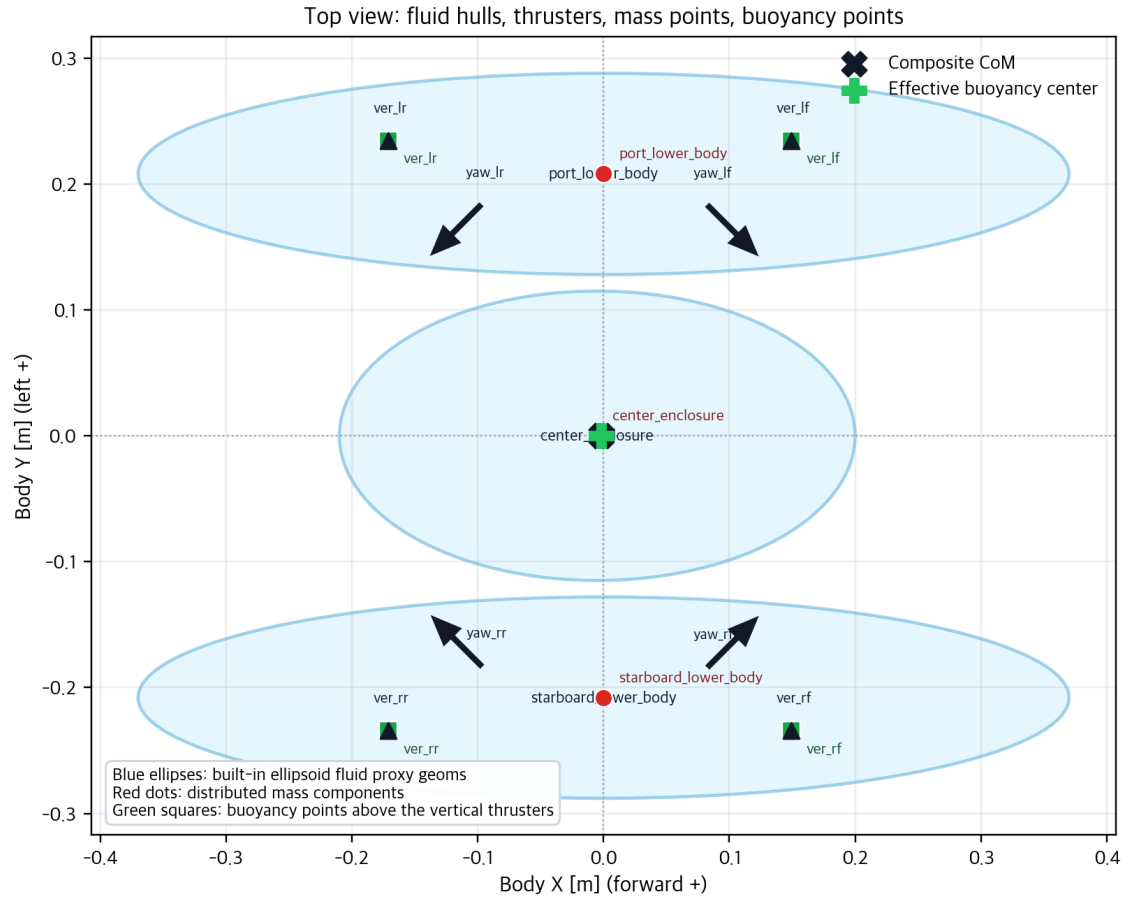


Figure 4: 상면도. 파란 타원은 built-in ellipsoid fluid proxy geom, 빨강은 질량 component, 초록은 부력 point, 검정 화살표는 yaw thruster 방향이다.

- 부력은 buoyancy point 또는 CoB offset을 통해 더 위쪽에서 작용한다.
- 질량중심 CoM과 부력중심 CoB, fluid drag center, thruster plane이 어긋나면 **depth hold가 pitch moment**를 만든다.
- 따라서 “뒤로 미끄러진다”는 현상은 유체엔진 문제라기보다 **수직 추력이 body attitude를 통해 수평 성분으로 새는 trim 문제**인 경우가 많다.

7 6DOF에서 어떤 물리력이 어떤 축에 들어가는가

7.1 먼저 body 축과 상태벡터를 읽어야 한다

runtime이 쓰는 기본 상태벡터는

$$\nu = [u, v, w, p, q, r]^T \quad (1)$$

이다. 여기서

- u, v, w 는 body frame 선속도(surge, sway, heave)
- p, q, r 는 body frame 각속도(roll, pitch, yaw)

를 뜻한다. 즉 현재 simulator의 hydrodynamics는 결국 **6자유도 wrench**

$$\tau = [F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]^T \quad (2)$$

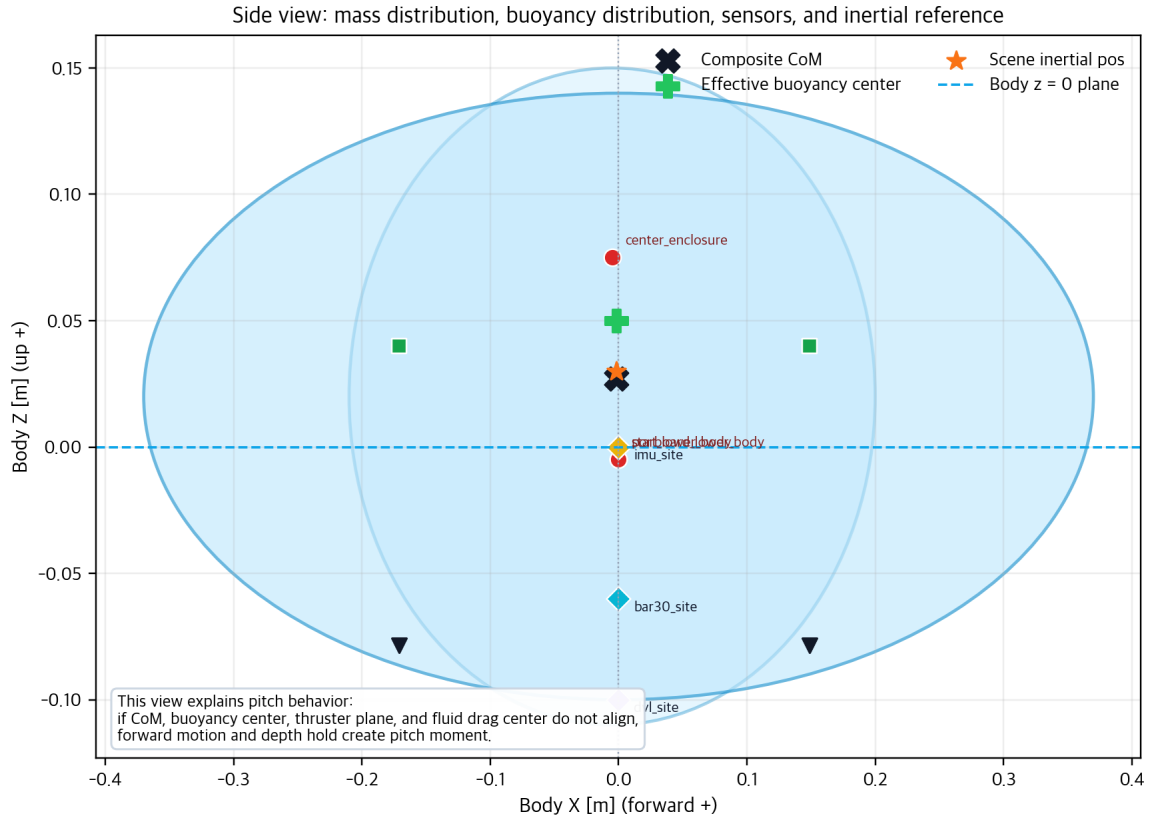


Figure 5: 측면도. 질량, 부력, vertical thruster plane, 센서 위치, inertial reference를 함께 표시했다. pitch bias가 생기는 이유를 설명할 때 가장 직접적인 그림이다.

를 어떻게 만들 것인가의 문제로 볼 수 있다.

즉 runtime의 모든 계산은 결국 다음 여섯 채널을 누가 얼마나 만드는가의 문제로 환원된다.

- 선형 힘 3개: surge, sway, heave
- 회전 모멘트 3개: roll, pitch, yaw

7.2 custom 경로에서는 어떤 힘을 합쳐서 최종 wrench를 만드는가

현재 `run_urdf_full.py` 의 custom 경로는 개념적으로 다음 합을 만든다.

$$\tau_{\text{total}} = \tau_{\text{buoyancy}} + \tau_{\text{surface_heave}} + \tau_{\text{added mass}} + \tau_{\text{damping}} + \tau_{\text{thruster reaction}} \quad (3)$$

여기서 더 핵심적인 custom 6-DOF hydrodynamic 항은

$$\tau_{\text{hydro}} = -M_A \dot{\nu}_r - C_A(\nu_r) \nu_r - D_1 \nu_r - D_2 |\nu_r| \odot \nu_r \quad (4)$$

형태다.

- M_A : added-mass diagonal
- $C_A(\nu_r)$: added-mass Coriolis matrix
- D_1 : 선형 damping

3D 6-DOF wrench view: the simulator finally builds these six quantities

$\tau = [F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]^T$ and each runtime term contributes to one or more of these six channels.

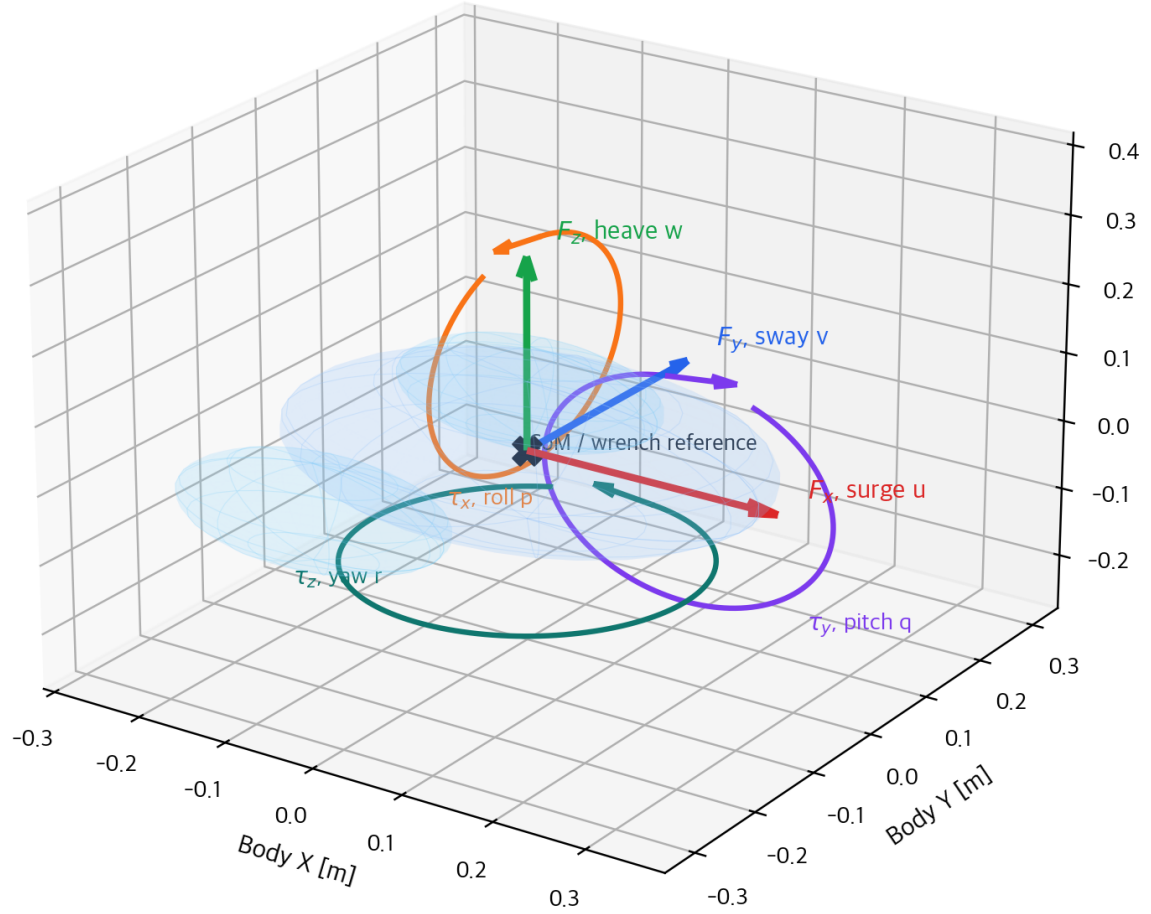


Figure 6: 3D wrench view. simulator가 최종적으로 만들어 내는 여섯 개 채널, 즉 F_x , F_y , F_z , τ_x , τ_y , τ_z 를 입체적으로 표현한 그림이다.

- D_2 : 2차 damping
- ν_r : 상대 속도(body velocity에서 water current를 뺀 값)

즉 custom 경로는 “drag를 하나 넣었다” 수준이 아니라, **관성형 항, Coriolis형 항, 선형 감쇠, 2차 감쇠, hydrostatics**를 합쳐서 body wrench를 만든 뒤, 마지막에 world frame으로 돌려 xfrs_applied에 더하는 구조다.

7.3 어떤 항이 어떤 자유도에 주로 작용하는가

이 그림을 읽는 법은 다음과 같다.

- **Distributed buoyancy force** 는 주로 heave(F_z)에 들어간다.
- **Buoyancy torque from point offset** 은 roll/pitch torque에 주로 들어간다.
- **Surface heave damping** 은 수면 근처 z축 chatter를 줄이기 위한 항이다.
- **Added mass, added-mass Coriolis, linear damping, quadratic damping** 은 6개 축 모두에 들어갈 수 있다.
- **Yaw torque scale on yaw thrusters** 는 사실상 yaw 축 authority만 키우는 보조 항이다.

Body axes and 6-DOF states used by the simulator

All runtime hydrodynamic terms are built around this body-frame ordering: $[u \ v \ w \ p \ q \ r]$.

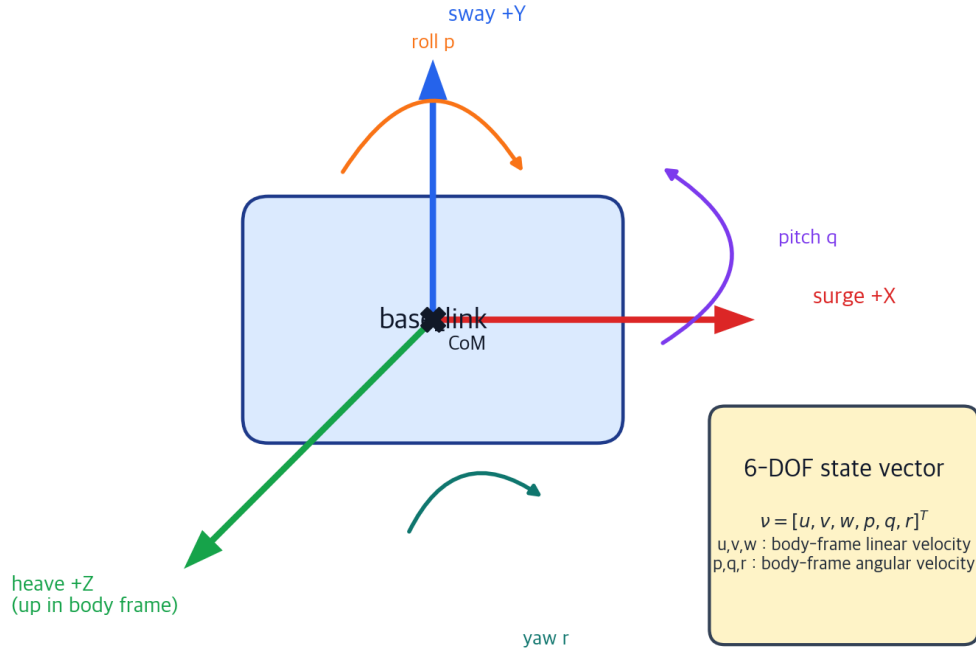


Figure 7: 현재 runtime이 쓰는 body frame 6-DOF 상태 정의. hydrodynamics helper 함수와 added-mass Coriolis 계산은 모두 이 축 정의를 기준으로 한다.

7.4 이전 custom baseline과 현재 ellipsoid 모델은 무엇이 다르게 커지는가

현재 runtime에서 두 경로는 다음 차이를 가진다. 여기서 읽는 순서는 “custom과 ellipsoid가 둘 다 현재 주력이다”가 아니라, **custom은 이전 baseline, ellipsoid는 현재 active 모델**이라는 전제를 두는 편이 맞다.

항목	custom	ellipsoid	설명
distributed mass/inertia synthesis	on	on	두 경로 모두 <code>body_components</code> 로 합성 mass/CoM/inertia를 다시 쓴다.
distributed buoyancy force	on	on	두 경로 모두 buoyancy point 또는 component point 에서 부력을 나눠 적용한다.
buoyancy torque from offset	on	on	CoM과 buoyancy point 차이로 roll/pitch restoring torque를 만든다.
surface heave damping	on	on	수면 근처 z축 chatter를 줄이기 위한 항이다.
full-submerged heave damping	off	on	현재 ellipsoid 경로에만 완전 잠수 구간 heave damping 보정이 추가된다.
added-mass inertia / Coriolis	on	off	custom에서만 Python이 직접 계산한다.
linear / quadratic damping wrench	on	off	custom에서만 body-frame 6-DOF damping wrench 를 직접 계산한다.
built-in ellipsoid fluid proxy force	off	on	ellipsoid 경로에서는 MuJoCo가 scene의 proxy geom 들로 유체력을 계산한다.
thruster reaction torque	on	on	스러스터 회전반작용 보정은 두 경로 모두 들어간다.
yaw torque scale	optional	on	현재 설정에서는 ellipsoid만 <code>yaw_torque_scale=4.0</code> 를 쓴다.

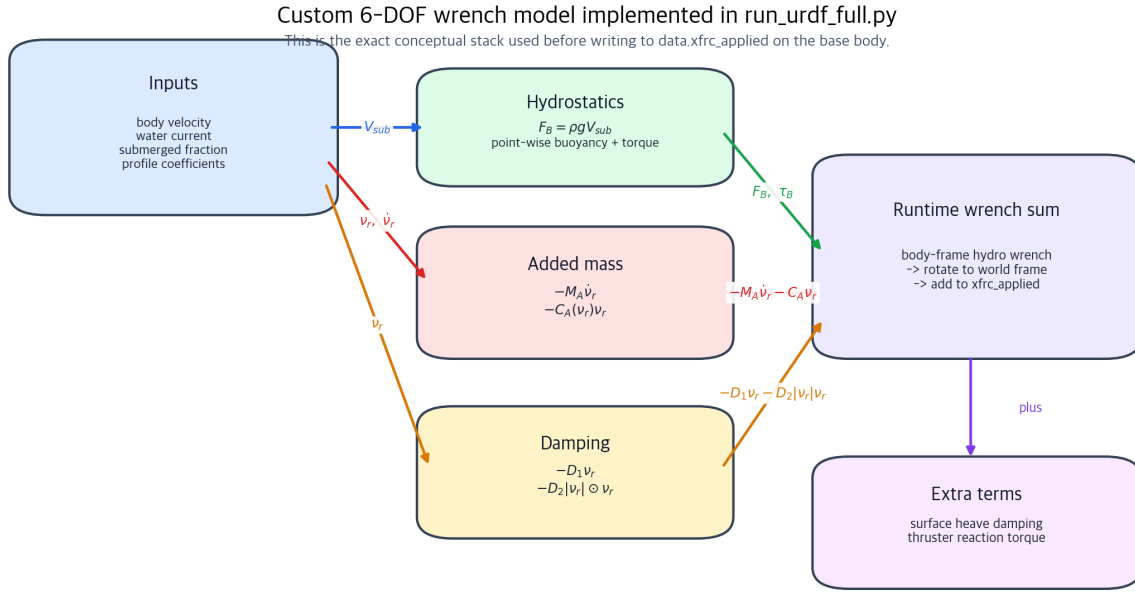


Figure 8: custom 6-DOF wrench pipeline. 실제 코드가 `data.xfrc_applied`에 쓰기 직전까지 어떤 항을 합치는지 개념적으로 정리한 그림이다.

즉 **현재 모델인 ellipsoid** 경로는 “scene만 믿는 순수 built-in mode”가 아니고, MuJoCo built-in ellipsoid fluid 위에 runtime 보정 항을 덧붙인 하이브리드 모델이다. 반대로 **custom** 경로는 **현재 운영 모델**이라기보다 **이전 Python 직접 계산형 baseline**으로 보는 편이 맞고, rigid-body integration과 contact, sensor frame, visualization은 여전히 MuJoCo가 맡는다.

8 질량, 관성, 부력은 현재 어떻게 구현되어 있는가

8.1 현재 질량 모델

현재 두 profile 모두 질량 component는 동일하다.

Component	mass [kg]	mass_pos x [m]	mass_pos z [m]
center_enclosure	6.0	-0.005	0.075
port_lower_body	4.5	0.000	-0.005
starboard_lower_body	4.5	0.000	-0.005
Total	15.0	composite CoM = (-0.002, 0.000, 0.027)	

즉 **중앙 상부에 6kg, 좌우 하부에 각각 4.5kg**이 있다고 생각하면 된다. 이 구조는 “상부 인클로저 + 좌우 lower body”라는 실물 직관과 맞고, 따라서 초보자도 왜 roll/pitch 응답이 그런지 상상하기 쉽다.

8.2 현재 관성 모델

여기서 중요한 함정이 하나 있다. 현재 `tank_ellipsoid_scene.xml`의 `diaginertia`는

```
diaginertia="0.001 0.001 0.001"
```

Runtime force/torque term map: what acts on which DOF, and in which mode

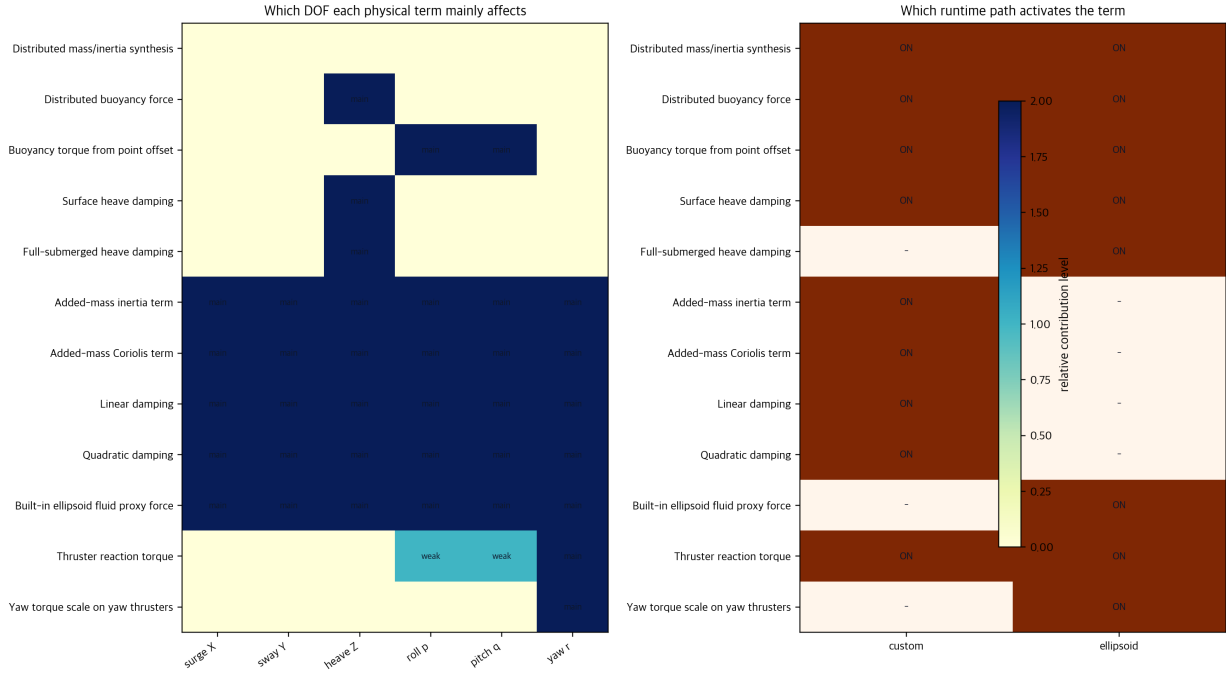


Figure 9: 왼쪽은 각 물리 항이 어떤 DOF에 주로 작용하는지, 오른쪽은 custom / ellipsoid 경로 중 어느 경로에서 그 항이 실제로 켜지는지를 정리한 matrix다.

로 매우 작다. 그러나 이 값이 최종 물리 모델의 전부는 아니다. 이 값은 **MuJoCo XML 로더가 scene을 받아들이기 위한 최소 안전 값**이고, 실제 runtime에서는 `run_urdf_full.py`가 `body_components`로 합성한 관성에 `body_inertia_scale_xyz`를 곱해 다시 덮어쓴다.

현재 계산 결과는 다음과 같다.

구분	I_{xx}	I_{yy}	I_{zz}
합성 raw inertia	0.473376	0.370230	0.739326
custom runtime inertia	0.473376	0.370230	0.739326
ellipsoid runtime inertia	0.000473	0.000370	0.000739

왜 이렇게 되느냐면, 현재 ellipsoid profile에서는

`body_inertia_scale_xyz = [0.001, 0.001, 0.001]`

이기 때문이다. 즉 **회전 관성은 원래 합성값의 0.1%만 남긴** 극단적 상태다. 이 값은 yaw를 매우 빠르게 만들 수 있지만, 동시에 damping과 controller가 과하면 “관성은 거의 없는데 딱딱하게 붙잡는” 느낌을 만들기도 한다.

8.3 현재 부력 모델

부력 기본식은 그대로다.

$$F_B = \rho g V_{\text{sub}} \cdot \text{buoyancy_scale} \quad (5)$$

하지만 이 문서에서 더 중요한 건 **어디에, 어떤 곡선으로, 어떤 비율로** 적용되느냐이다.

- 잠김비 모델: linear가 아니라 ellipsoid

- 부력 point: vertical thruster 위 4점
- point share: 각 0.25
- x/z offset: cob_x_offset=0.009, cob_z_offset=0.010
- buoyancy_scale: 1.008

즉 현재 유효 부력중심은 대략

$$(-0.002, 0.0001, 0.050) \text{ m} \quad (6)$$

정도이며, composite CoM의

$$(-0.002, 0.0000, 0.027) \text{ m} \quad (7)$$

보다 약 23mm 위에 있다. 따라서 roll/pitch에 대해서는 restoring moment가 생기지만, yaw에 대해서는 직접 restoring이 거의 없다.

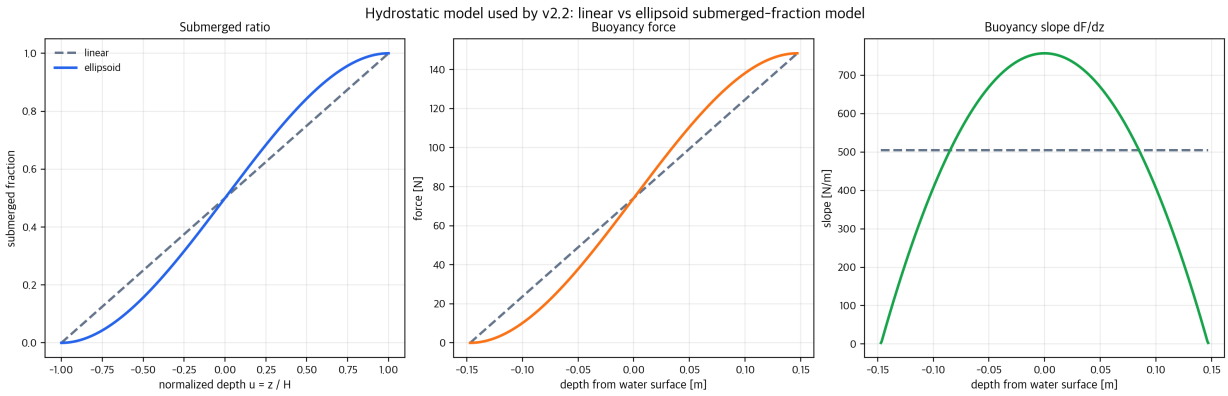


Figure 10: 현재 코드가 쓰는 잠김비/부력 모델. 선형 모델과 비교하면 ellipsoid 모델은 수면 중앙 근처에서 기울기가 더 크고, 양 끝에서는 더 부드럽게 포화된다.

이 그림이 의미하는 바는 명확하다.

- 수면 근처에서는 작은 깊이 변화에도 부력 변화가 민감하다.
- 완전 잠수/완전 노출 근처에서는 기울기가 줄어들어 chatter를 줄인다.
- 따라서 z축 진동을 설명할 때 “부력이 선형이나 비선형이나”는 의미가 있다.

9 스러스터 모델은 현재 어떻게 구현되어 있는가

9.1 현재 공통 thruster knob

thruster_params.json 의 현재 공통 값은 다음과 같다.

항목	현재 값
deadzone	0.002
tau_up	0.04
tau_down	0.06
reverse_asymmetry	0.70
reaction_torque_gain	0.0012
gain_scale_all	2.6
command_limit	0.90

이 값들이 의미하는 바는 다음과 같다.

- deadzone: 스틱을 살짝 건드렸을 때 모터가 바로 움직이지 않게 하는 구간
- tau_up/down: 힘이 갑자기 바뀌지 않고 1차 지연으로 따라오게 하는 시간상수
- gain_scale_all: 전체 추력을 한 번에 키우거나 줄이는 전역 배율
- reverse_asymmetry: 후진 추력이 전진보다 약한 현실을 반영하는 계수

9.2 실제 추력 생성 경로

runtime에서 스터스터 명령은 다음 순서로 변환된다.

1. pilot 또는 FCU 명령이 정규화된 $[-1, 1]$ command로 들어온다.
2. `shape_thruster_command()` 가 deadzone과 command limit를 적용한다.
3. `first_order_response()` 가 tau_up/down에 따라 상태를 부드럽게 따라가게 만든다.
4. measured T200 curve가 활성화돼 있으면 PWM-force curve를, 아니면 polynomial fallback force를 쓴다.
5. 마지막으로 per-thruster gain과 global gain을 곱해 실제 force가 된다.

이 그림은 조종감을 설명할 때 가장 중요하다.

- 저입력 구간이 너무 무디면 deadzone이나 polynomial 초반 기울기를 봐야 한다.
- 입력은 빠르는데 실기보다 부드러우면 tau_up/down 을 늘린다.
- 같은 제어기인데도 전체가 강하면 gain_scale_all 을 낮춘다.

10 이전 custom baseline과 현재 ellipsoid 모델의 hydrodynamics는 어떻게 다른가

10.1 이전 custom baseline

custom 경로는 이전에 쓰던 기준 모델로, Python runtime이 직접 계산한 hydrodynamic wrench를 `xfrc_applied` 로 body에 주입한다. 여기에 들어가는 항은 크게 네 종류다.

- hydrostatic buoyancy
- distributed buoyancy torque
- added mass / added-mass Coriolis
- linear / quadratic damping

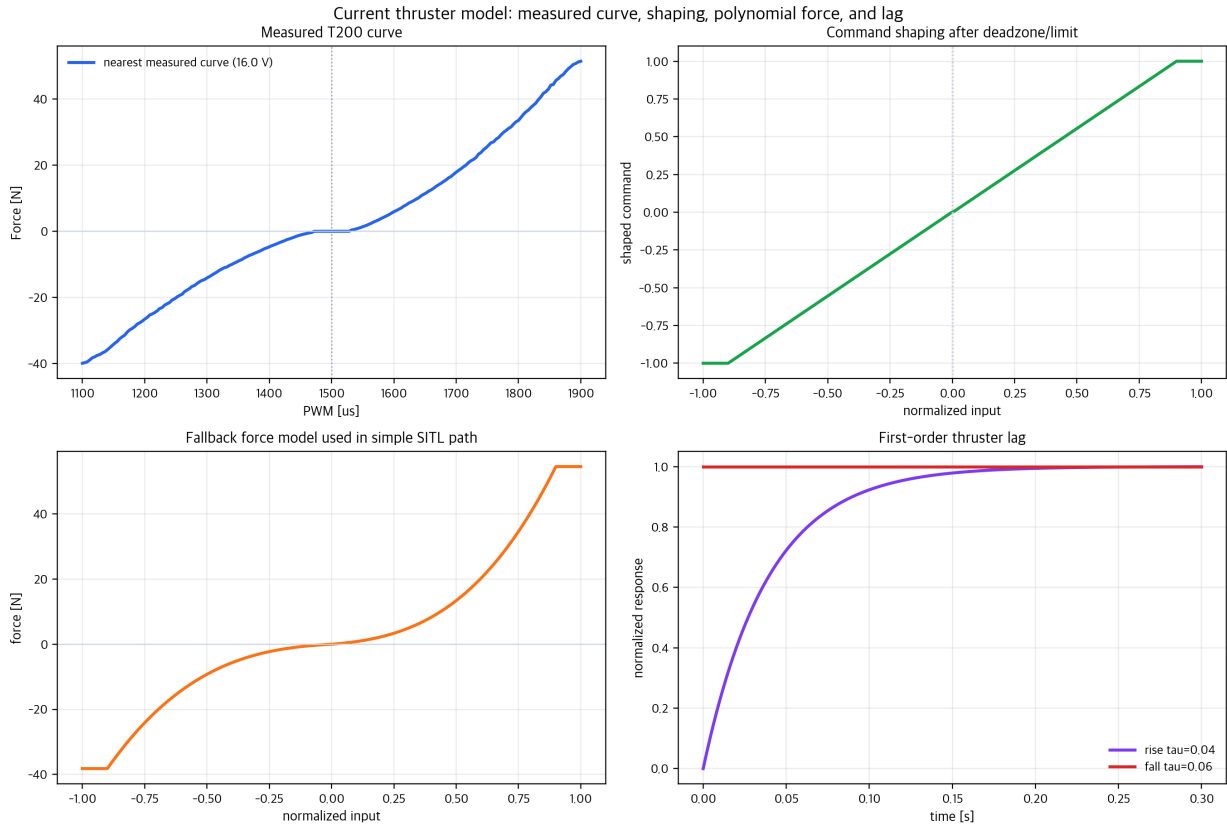


Figure 11: 현재 스러스터 모델. measured T200 curve, deadzone/limit shaping, polynomial fallback force, first-order lag을 함께 보여 준다.

10.2 현재 ellipsoid 모델

ellipsoid 경로는 **현재 모델**이다. scene에 있는 세 개의 fluid proxy geom이 MuJoCo built-in ellipsoid fluid를 계산하고, 그 위에 runtime에서 분산 질량, 분산 부력, heave damping, reaction torque, yaw torque scale 같은 보정이 더해진다. 현재 scene의 대표 값은 다음과 같다.

Geom	position [m]	size [m]	fluidcoef
fluid_center_enclosure	(-0.005, 0.000, 0.020)	(0.205, 0.115, 0.130)	(0.620, 0.175, 1.050, 0, 0)
fluid_port_lower_body	(0.000, 0.208, 0.020)	(0.370, 0.080, 0.120)	(0.520, 0.150, 0.750, 0, 0)
fluid_starboard_lower_body	(0.000, -0.208, 0.020)	(0.370, 0.080, 0.120)	(0.520, 0.150, 0.750, 0, 0)

10.3 profile에서 runtime으로 파생되는 hydrodynamic 값

현재 ellipsoid profile의 핵심 값은 아래처럼 읽으면 된다.

- `effective_cd_linear=[0.100,0.095,0.145]`
- `effective_cd_angular=[2.70,1.30,0.005]`
- `added_mass_scale_linear=[0.45,0.28,0.65]`
- `added_mass_scale_angular=[1.00,0.92,1.02]`
- `linear_damping_ratio_linear=2.0`
- `linear_damping_ratio_angular=4.2`

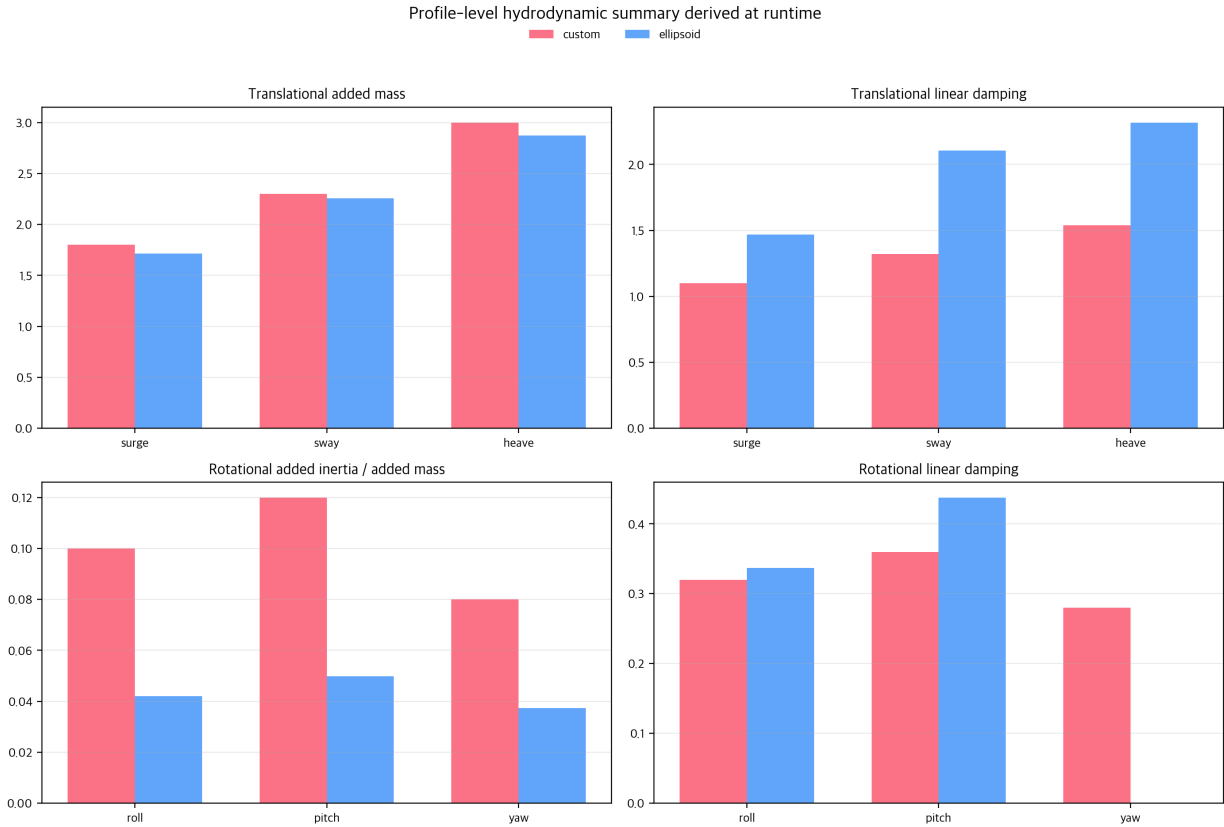


Figure 12: runtime이 실제로 쓰는 hydrodynamic 계수 요약. 이전 baseline인 custom 과 현재 모델인 ellipsoid 가 added mass와 damping을 어떻게 다르게 쓰는지 보여 준다.

여기서 특히 눈여겨볼 것은 **yaw angular drag의 유효 계수**와 **yaw torque scale** 이다. 현재 `effective_cd_angular[2]` 는 매우 낮고, 대신 `yaw_torque_scale=4.0` 이라서 물리적으로는 yaw 저항을 낮추고, 추진 authority는 크게 키운 상태다.

11 왜 MANUAL yaw와 DEPTH_HOLD yaw 느낌이 다를 수 있는가

초보자가 가장 헷갈리는 지점이 여기다. 같은 yaw stick인데 mode만 바꾸면 느낌이 달라지는 이유는, assisted mode에서는 yaw가 더 이상 direct motor command가 아니기 때문이다.

현재 injected SITL yaw 관련 파라미터는 다음과 같다.

파라미터	현재 값
RC4_DZ	10
ACRO_YAW_P	8.00
ATC_ANG_YAW_P	2.20
ATC_ACC_Y_MAX	2200.0
ATC_RATE_Y_MAX	360.0
ATC_RAT_YAW_P/I/IMAX/D	0.100 / 0.012 / 0.12 / 0.0025

즉 assisted mode yaw는 다음 네 층이 곁해진 결과다.

1. RC deadzone과 stick scaling
2. desired yaw rate 생성
3. FCU yaw rate loop

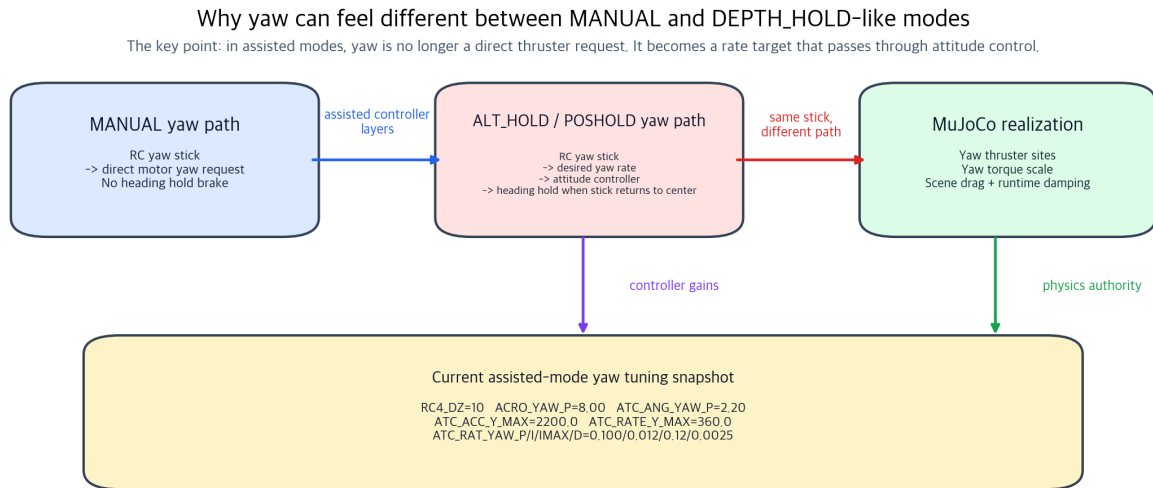


Figure 13: MANUAL yaw와 assisted yaw의 차이. assisted mode에서는 yaw stick이 먼저 desired yaw rate로 바뀌고, 그다음 attitude controller를 거친다.

4. MuJoCo 쪽 actual yaw torque authority와 drag

그래서 “yaw가 느리다”고 느껴질 때는 반드시 어느 층이 병목인지를 분리해야 한다.

12 현재 측정 결과는 무엇을 말해 주는가

12.1 실제 mode path 비교

이 repository에는 이전 baseline인 custom과 현재 모델인 ellipsoid 두 경로에 대해 실제로 재생한 ros2bag 요약 JSON이 이미 들어 있다. 여기서는 그 결과를 새 그림으로 다시 묶었다.

이 결과가 의미하는 바는 다음과 같다.

- 이전 baseline인 custom은 authority가 큰 대신 path가 더 길게 늘어지는 경향이 있다.
- 현재 모델인 ellipsoid는 이동량은 보수적이지만 turn error와 자세 disturbance가 더 작게 정리되는 경향이 있다.
- 따라서 보고서에서 “현재 모델”이라고 부를 것은 ellipsoid 쪽이고, custom은 과거 모델 또는 비교용 baseline으로 설명하는 편이 맞다.

12.2 저장돼 있는 archived engine snapshot

기존 보고서에 들어가 있던 step-test 요약도 repository 안에 남아 있다. 이 값은 현재 mode-path 측정과는 별개의, **archived force/moment snapshot**이다.

이 그림은 딱 세 가지를 설명할 때 유용하다.

- current custom은 가장 공격적인 이전 계열 모델이다.
- current ellipsoid는 지금 설명의 기준이 되는 현재 모델이다.
- legacy custom과 current custom은 모두 현재 모델 그 자체라기보다, ellipsoid로 오기 전 계열을 설명하는 비교 기준으로 읽는 편이 맞다.

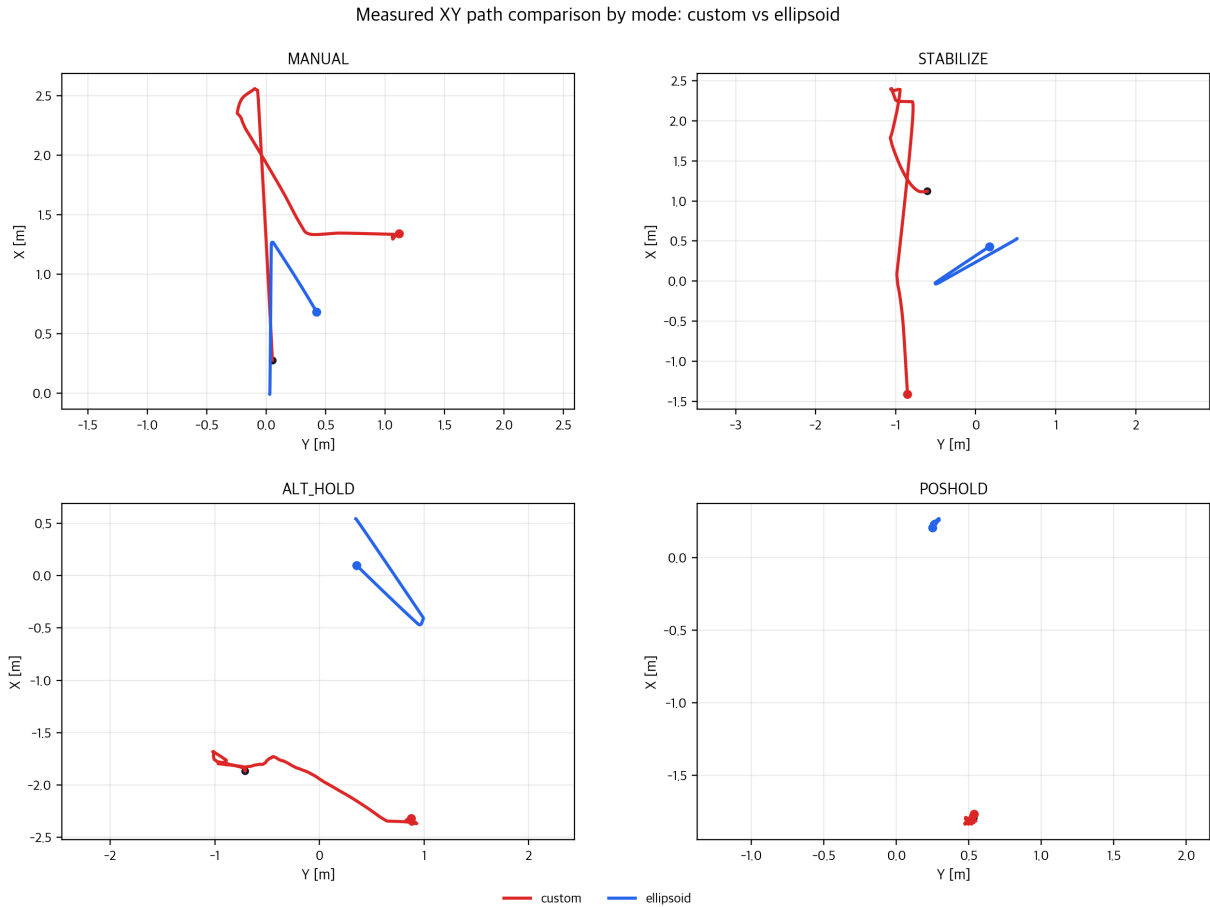


Figure 14: mode별 measured XY path. custom과 ellipsoid가 같은 “전진 → 90도 turn → 우측 이동” 시퀀스를 어떻게 다르게 누적하는지 바로 볼 수 있다.

13 현재 문제를 볼 때 어디를 먼저 보면 되는가

증상	먼저 볼 파일	먼저 볼 파라미터
yaw가 MANUAL보다 DEPTH_HOLD에서 둔함	start_ardusub_sitl_mj311.sh	RC4_DZ, ACRO_YAW_P, ATC_RAT_YAW_*
yaw는 빠르는데 금방 딱 멈춰 버림	sim_profiles.json, start_ardusub_sitl_mj311.sh	effective_cd_angular[2], ATC_ANG_YAW_P, ATC_RAT_YAW_D
depth hold 중 뒤로 밀림	sim_profiles.json, tank_ellipsoid_scene.xml	cob_x_offset, buoyancy point 위치, fluid proxy z 위치
전진 시 과하게 숙여짐	tank_ellipsoid_scene.xml	fluid proxy z, CoM/CoB 상대 위치, pitch damping
위아래로 hunting	thruster_params.json, start_ardusub_sitl_mj311.sh, sim_profiles.json	tau_up/down, PSC_D_*, heave_damping_scale
전체가 너무 둔함	thruster_params.json, sim_profiles.json	gain_scale_all, deadzone, added_mass, effective_cd
병진은 좋은데 yaw만 약함	sim_profiles.json, run_urdf_full.py	yaw_torque_scale, yaw thruster gain, yaw drag

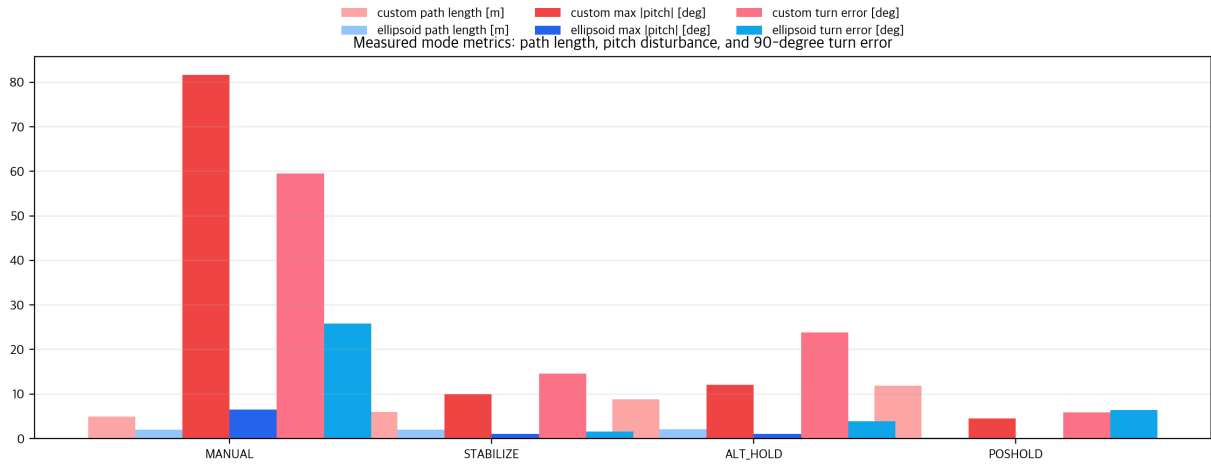


Figure 15: path length, 최대 pitch disturbance, 90도 turn error 비교. 숫자만 보는 것보다 어떤 경로가 더 정리되는지 훨씬 직관적으로 드러난다.

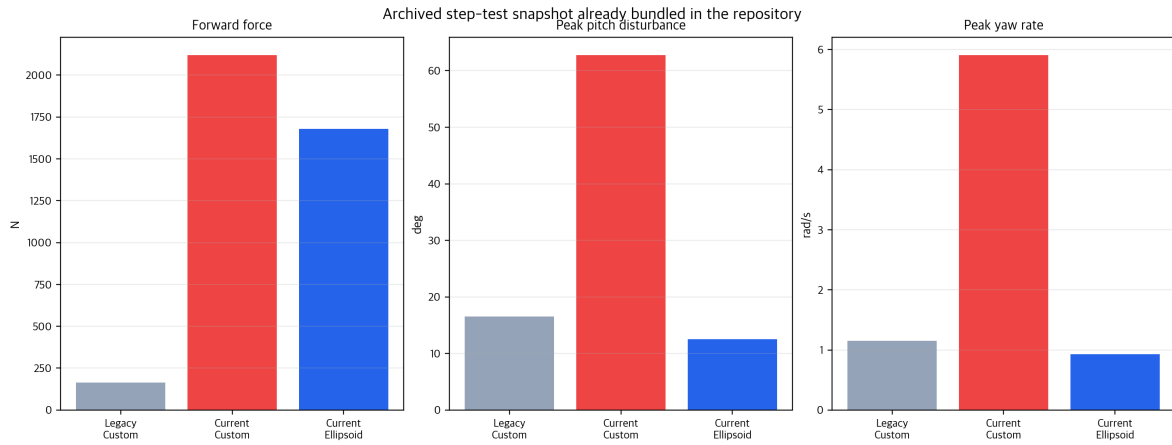


Figure 16: repository에 저장돼 있는 archived engine snapshot. legacy custom, current custom, current ellipsoid의 step-test 지표를 세 축으로 다시 보여 준다.

14 실행과 재현 방법

14.1 권장 실행

```
cd /Users/kanghyunmin/Desktop/uuv_sim/uuv_mujoco/v2.2
./reset_uuv_sim.sh --with-qgc-stop --wipe-eprom
./start_sitl_mujoco_mj311.sh \
  --ros2-real-pkg-compat \
  --sitl-no-rebuild \
  -- \
  --tank-549x274x132 \
  --fluid-model ellipsoid \
  --profile ellipsoid \
  --no-qgc-video
```

14.2 새 문서 자산 생성

```
cd /Users/kanghyunmin/Desktop/uuv_sim
/Users/kanghyunmin/.venvs/mujoco311/bin/python \
  document/docsource/generate_uuv_v22_explainer_assets.py
```

14.3 새 PDF 빌드

```
cd /Users/kanghyunmin/Desktop/uuv_sim/document/docsource
xelatex -interaction=nonstopmode uuv_mujoco_v22_explainer_kr.tex
xelatex -interaction=nonstopmode uuv_mujoco_v22_explainer_kr.tex
```

15 이 문서만 보고 바로 설명할 때의 말하기 순서

만약 발표나 회의에서 3분 안에 설명해야 한다면, 아래 순서를 그대로 쓰면 된다.

1. “이 시스템은 ArduSub가 제어를 담당하고, MuJoCo runtime이 실제 힘과 센서를 담당하는 구조다.”
2. “물리는 `sim_profiles.json`, `thruster_params.json`, `tank_ellipsoid_scene.xml` 세 파일이 핵심이다.”
3. “질량은 3분할, 부력은 vertical thruster 위 4점 분산이다.”
4. “custom은 이전 baseline이고, 현재 모델은 built-in ellipsoid fluid에 runtime 보정을 얹은 ellipsoid 경로다.”
5. “실측 기준으로는 현재 ellipsoid 모델이 더 보수적이지만 더 정리되고, custom 계열은 이전 비교 기준으로 남아 있다.”
6. “문제가 생기면 먼저 증상별로 어느 층이 병목인지 분리해서 본다.”

16 결론

현재 `uuv_mujoco/v2.2` 는 더 이상 실험 파일이 뒤섞인 폴더가 아니라, **현재 모델(ellipsoid)과 이전 baseline(custom)을 함께 보존한 상태**다. 그리고 이 문서의 중요한 목적은 “물리식을 많이 쓰는 것”보다, **지금 코드에서 어떤 파일이 어떤 현상을 만들고 있는지를** 한 번에 설명 가능하게 만드는 데 있다.

핵심만 다시 정리하면 이렇다.

- scene XML은 위치와 shape를 준다.
- profile JSON은 부력, drag, added mass, 관성 스케일을 준다.
- thruster JSON은 입력을 힘으로 바꾸는 방식을 준다.
- SITL script는 FCU 제어 gain을 준다.
- runtime Python은 이 모든 것을 실제 wrench로 합성한다.
- 현재 모델은 ellipsoid 경로이며, MuJoCo built-in fluid 위에 runtime 보정 항을 더한 구조로 이해해야 한다.
- custom 은 현재 모델의 경쟁안이라기보다, 이전 계열 baseline으로 설명하는 것이 맞다.

따라서 누가 “왜 이 로봇이 이렇게 움직이느냐”고 물으면, 이제는 감으로 답하지 않고 **scene, profile, thruster, FCU, runtime** 다섯 층으로 나눠 설명하면 된다. 이 문서는 바로 그 설명을 위해 만들어졌다.